Administration & Tuning des Bases de données

Dr FERRAHI Ibtisam
Maître de conférence, USTHB

```
create or replace trigger TOTAL_INTERV_Insert_Delete
after
insert or delete
on intervenants
for each ROW
BEGIN
CASE
WHEN INSERTING THEN update employe
set TOTAL_INTERVENTIONS= TOTAL_INTERVENTIONS+1
where numemploye= :new.numemploye;
WHEN DELETING THEN  update employe
set TOTAL_INTERVENTIONS= TOTAL_INTERVENTIONS-1
where numemploye= :old.numemploye;
END CASE;
EXCEPTION
WHEN OTHERS then  DBMS_OUTPUT.PUT_LINE('error : '||sqlcode||' '||sqlerrm);
END;
/
```

Insert into intervenant values

Delete from intervenant

2

4

Contraintes d'intégrité

3

Introduction

- **Les contraintes d'intégrité** permettent d'assurer que les données d'une BD sont conformes à la réalité qu'elles représentent
- Une contrainte d'intégrité est une règle qui peut être **simple** ou **complexe** pouvant prendre la valeur vraie ou faux
  - Elle s'applique sur les données d'une base de données
  - Elle permet de maintenir la cohérence de données

4

Introduction

Quand les déclarer ?

- A la création de la base de données.
- A tout instant de l'existence d'une base de données, on doit pouvoir définir une nouvelle contrainte d'intégrité de manière dynamique. Le SGBD doit fournir des mécanismes pour vérifier que la base est cohérente vis à vis de cette contrainte et l'accepter ou la rejeter.

Comment ?

- A l'aide d'instructions ou de programmes SQL, ou ses différentes extensions SQL 86, SQL 89, PL/SQL etc.
- Le SGBD doit fournir des mécanismes pour vérifier que la base est cohérente vis à vis de cette contrainte et accepter ou rejeter toute mise à jour de la BD.

5

Introduction

- Soit la relation

  Employés (**NSS**, Nom, Date_recrutement, Fonction, code_service*)

- **À la création**
  - **CREATE TABLE** Employés (NSS integer PRIMARY KEY, Nom char (40) NOT NULL, date_recrutement date, code_service varchar (30) , **CONSTRAINT** C1 **CHECK** (Fonction in ('Ingenieur' 'comptable'))
- **Au cours de la vie de la BD**
  - **ALTER TABLE** Employés **ADD CONSTRAINT** C2 **CHECK** (Fonction **IN** ('Ingenieur' 'comptable'))

6

Les types de contraintes d'intégrités

- **Les contraintes d'intégrité statiques** sont celles qui correspondent à un prédicat sur l'état courant de la base.
- **Les contraintes d'intégrité dynamiques** sont celles qui concernent le passage d'un état à un autre.

Les types de contraintes d'intégrités

Contrainte		Individuelle: *Un tuple, Simple*		Ensembliste: *Plusieurs tuples, Complexe*	
		Statique *Un état*	**Dynamique** *Changement d'état*	Statique	**dynamique**
Intra relation: ***Une table***	Mono attribut *Une colonne*	NSS char(12) Unicité clé	Point indiciare>= valeur minimale	Not Null	Salaire employé ne peut pas dépasser 2 fois la moyenne des salaires
	Multi attribut *Plusieurs colonnes*	Unicité d'une clé composée	Date_recrutement < date_promotion	Unicité de clé composée	Somme des crédits < somme des débits pour un compte
Inter relation ***Plusieurs tables***	Multi attribut			Clé étrangère	Quantité commandée<quantité en stock

Les contraintes d'intégrité statiques

Les contraintes statique intra relation

- **CI individuelles intra relation mono attribut**
  - **Unicité de la clé**
    - **Plage de valeurs :** 4.000 ≤ salaire ≤ 20.000, : salaire between 10000 and 20000
    - **Liste de valeurs :** Couleur dans [bleu, rouge, vert] : couleur IN ('bleu', 'rouge', 'vert')
    - **Contraintes de format** : NOM CHAR(20)

- **CI individuelles intra relation multi attribut**
  - **Contraintes entre constituants :** Dépenses< recettes

10

Les contraintes statique intra relation

- **L'instruction ASSERT**
  - **L'instruction ASSERT permet de définir une contrainte d'intégrité générale**

    ASSERT *<nom assertion> [on relation] : prédicat ;*

    Create assertion *« nom assertion »* ***Check*** *(expression-conditionnelle);*

- **Exemple**
  - **ASSERT** CI1 **ON ETUDIANT** : Specialité IN ('SSI', 'SII', 'MIV') ;
  - **Create assertion** CI2 **Check Not exists** (select * from Etudiants
    Where (specialité = 'Data sciences'))

11

Les contraintes statiques SQL

- **Niveau Colonne et niveau Table**

Niveau de contrainte	Description
Colonne	Référencie une seule colonne et se définit dans la spécification de la colonne: applicable à n'importe quel type de CI
Syntaxe	Column [CONSTRAINT *constraint_name] constraint_type*
Table	Référencie une ou plusieurs colonnes et se définit indépendamment de la définition des colonnes: applicable à toutes les commandes sauf NOT NULL
Syntaxe	Column, … [CONSTRAINT *constraint_name*] *constraint_type (column, …)*

- **Syntaxe :**

Constraint_name nom de la contrainte
Constraint_type type de la contrainte

12

Les contraintes d'intégrité statique ensembliste

- **Niveau Colonne SQL**
  - **NOT NULL:** interdit la présence des valeurs null dans la colonne
  - **Exemple:**
    - **CREATE TABLE Employés (**
      NSS number(4),
      Nomemp varchar(10) not null,
      Numproj number (9),
      Numdept number(2) not null **) ;**
    - La contrainte **NOT NULL** s'applique aux colonnes **Nomemp et Numdept** de la table **Employés.**
    - On pourrait écrire: Numdept **number** (2) **Constraint** *C1* **NOT NULL**;

13

Les contraintes d'intégrité statique ensembliste

- **Niveau Colonne et niveau Table**
  - La contrainte de clé unique se définit au niveau table ou au niveau colonne
  - **Exemple:**

CREATE TABLE departement (Numdept number (2),
Nomdept varchar(12) Constraint *departement_nomdept_uk* UNIQUE);

  - On aurait pu écrire Nomdept UNIQUE varchar(12)
  - Constraint *departement_nomdept_uk* UNIQUE(Nomdept)

- **Remarque :** Oracle Server contrôle la colonne UNIQUE en créant implicitement un index unique sur la clé

14

Les contraintes d'intégrité statique ensembliste

- **Niveau Colonne et niveau Table**
  - **Primary key: NOT NULL + UNIQUE**
  - **Exemple:**
  - **CREATE Table** département
    (Numdept Number(2)
    Nomdept Varchar(14)
    Constraint *département_nomdept_uk* Unique (nomdept)
    Constraint *département_numdept_pk* PRIMARY KEY (Numdept)) ;

- **Remarque:** un index unique est crée automatiquement pour une colonne PRIMARY KEY

15

Les contraintes INTRA-RELATION multi attributs

- **Verticale**
  - Contrôler la valeur d'un constituant d'un n-uplet en fonction des valeurs de ce constituant pour les autres n-uplets.
- **Horizontale**
  - Contrôler la valeur d'un constituant en fonction des valeurs apparaissant dans les autres constituants

Les contraintes INTRA-RELATION multi attributs

- **Verticale**
  - Soit la relation **Employé** (Nom, Salaire, Nom-Rayon)
  - *« Un employé ne peut gagner plus du double de la moyenne des salaires de son rayon »*

ASSERT CI2 **ON** Employé E :
Salaire ≤ 2 * (**SELECT** AVG (Salaire)
FROM Employé
WHERE Nom-Rayon = E.Nom-Rayon);

- **Horizontale**
  - soit la relation **Rayon** (Nom-Rayon, Etage, Recettes, Dépenses, Nb-Employés)
  - Pour exprimer que les dépenses doivent être inférieures ou égales aux recettes, on écrira : **ASSERT** CI3 **On** Rayon : Dépenses ≤ Recettes

Les contraintes INTRA-RELATION multi attributs statiques

- Contrôler que l'ensemble des valeurs d'un constituant d'une relation R1 apparaît également dans une relation R2
- Exprimer le fait que tout rayon apparaissant dans la relation Employé est décrit dans la relation Rayon

ASSERT *CI5 (* ***SELECT*** *NOM-Rayon*
FROM *Employé) IS IN (****SELECT*** *Nom-Rayon*
FROM *Rayon);*

Est remplacée par la CI FOREIGN KEY

Les contraintes INTRA-RELATION multi attributs statiques

- **FOREIGN KEY**
  - La contrainte **FOREIGN KEY** définit une colonne d'une table par référence à une autre table
  - **Exemple :**

CREATE table employe (
Numemp number(4)
Nomemp varchar(10) NOT NULL
Numdept NUMBER(2) NOT NULL,
Constraint *employe_numdept_fk* **FOREIGN KEY** (numdept)
REFERENCES departement(numdept)

Les contraintes d'intégrité dynamiques

Les contraintes d'intégrités dynamiques

- Elles sont définies au cours de changements d'états.
- Il est possible de contrôler le passage d'un état à un autre.

- Les n-uplets du nouvel état dépendent de ceux de l'ancien:
  - Directement à l'aide d'assertions
  - Triggers ou déclencheurs

Les contraintes d'intégrités dynamiques

- **Contrainte d'intégrité dynamique intra-relation**
  - **Exemple** : le salaire d'un employé ne peut diminuer

```
ASSERT CI6 ON UPDATE OF Employé (Salaire)
NEW Salaire ≥ OLD Salaire;
```

22

Les contraintes d'intégrités dynamiques

- **Contrainte d'intégrité dynamique inter-relation**
  - **Exemple :** Soit la BD contenant les deux relations suivantes :

EMPLOYE (nom_emp, IDD, salaire)

DEPARTEMENT (IDD, nom_dept, directeur, nb_emp)

  - Et soit la contrainte d'intégrité :

« Tout employé du département 'Recherche' doit avoir un salaire supérieur à 150000 DA »

```
CREATE ASSERTION CHECK NOT EXISTS
 ( SELECT *
   FROM EMPLOYE
   WHERE salaire <= 150000  AND IDD= ( SELECT IDD
                                       FROM DEPARTEMENT
                                       WHERE nom_dept = 'Recherche')))
```

23

- **Modification des contraintes d'intégrités**
  - **Ajouter une CI**
    - **ALTER TABLE** <nom_table>
      ADD CONSTRAINT nom_contrainte type_contrainte;
  - **Supprimer une CI**
    - **ALTER TABLE** <nom_table>
      DROP CONSTRAINT nom_contrainte

24

Les contraintes d'intégrités dynamiques

- **Un trigger est une règle ayant la structure suivante :**
  - « si événement et/ou condition alors action »
  - L'action est déclenchée à la suite de l'événement, si la condition est vérifiée.

- **Exemple:** Lorsqu'un nouvel employé est inséré dans la base, le nombre d'employés figurant dans la relation rayon doit augmenter de un pour le rayon du nouvel employé

```
CREATE TRIGGER EMPINS
ON INSERTION OF Employé
( UPDATE RAYON
SET Nb-Employés = Nb-Employés + 1
WHERE Nom-Rayon = :NEW Employé.Nom-Rayon );
```


Création de triggers PL/SQL

```
CREATE TRIGGER [Nom_Trigger]
{BEFORE / AFTER}
{evenement :DELETE / INSERT/ UPDATE [OF liste_colonnes]}
ON Nom_Table
[REFERENCING OLD [AS] nom_ancienne_valeur
             / NEW [AS] nom_nouvelle_valeur
[FOR EACH {ROW /statement}
BEGIN .............
<<<< bloc d'instructions PL/SQL>>>>
END;
```

BLOC Evènement

BLOC Action
Une action est soit une vérification soit une mise à jour

Exemple:

- **Etudiant** (mat, nom,code_section) Evaluation (mat, codemodule , note)
- Section (codesection , nom, NB_etud)

L'action : Insert into etudinat values (10, 'ali', 'A');

```
CREATE TRIGGER T1
AFTER INSERT ON ETUDIANT
FOR EACH ROW
begin
  UPDATE section
  SET    nb_etud=nb_etud+1
  WHERE  codesection = :new.codesection;
END
```

Création de triggers PL/SQL

- **Create Trigger:** crée un nouveau déclencheur associé à la table spécifiée et exécute les actions précisées notamment dans < Nom_Trigger >, quand certains évènements surviennent.
- **Le NOM du Trigger doit être unique dans un même schéma**
- **Evènement:** Insert, Update, delete

Création de triggers PL/SQL

- **Définition de l'événement du trigger**
  - Elle comprend le type d'instruction SQL qui déclenche le trigger :
    - **DELETE, INSERT, UPDATE** On peut avoir une, deux ou les trois.
    - Pour **UPDATE**, on peut spécifier une liste de colonnes. Dans ce cas, le trigger ne se déclenchera que si l'instruction UPDATE porte sur l'une au moins des colonnes précisée dans la liste.
    - S'il n'y a pas de liste, le trigger est déclenché pour toute instruction UPDATE portant sur la table.

Création de triggers PL/SQL

- **Option BEFORE/AFTER** L'appel du déclencheur peut avoir lieu avant le traitement de l'opération ou après.
- **Option BEFORE/AFTER**
  - Elle précise le moment de l'exécution du trigger
  - **BEFORE :** le bloc action est levé avant l'exécution de l'événement
  - **AFTER :** le bloc action est levé après l'exécution de l'événement
- After: Toute modification, dont la dernière insertion, MAJ ou suppression est visible par le déclencheur.

31

Création de triggers PL/SQL

- **Types de triggers**
  - **Le type d'un trigger détermine :**
    - **Quand ORACLE déclenche le trigger,**
    - **Combien de fois** ORACLE déclenche le trigger.
  - **Le type du trigger est défini par l'utilisation de l'une ou l'autre des options suivantes :**
    - **BEFORE, AFTER, FOR EACH ROW**
    - **ORACLE propose deux types de triggers**
      - **les triggers lignes (row)** qui se déclenchent individuellement pour **chaque ligne de la table affectée par le trigger,**
      - **les triggers globaux (statement)** qui sont déclenchées une seule fois.
  - **Si l'option FOR EACH ROW est spécifiée, c'est un trigger ligne, sinon c'est un trigger global.**

32

Les triggers Lignes

- On peut introduire **une restriction** sur les lignes à l'aide d'une expression logique SQL: c'est la clause **WHEN** :
- Cette expression est évaluée pour chaque ligne affectée par le trigger.
- Le trigger n'est déclenché sur une ligne que si l'expression **WHEN** est vérifiée pour cette ligne.
-
- **Exemple:** WHEN (new.empno>0) empêchera l'exécution du trigger si la nouvelle valeur de EMPNO est 0, négative ou NULL.

33

Important:

- Si plusieurs déclencheurs sont définies pour **le même évènement**, ils sont déclenchés suivant l'ordre alphabétique de leurs noms.
- Select ne modifie aucune ligne donc aucun déclencheur ne peut être défini.
- Le bloc d'action
  - Il peut contenir du SQL et du PL/SQL.
  - Il est exécuté si l'instruction de déclenchement se produit et si la clause de restriction **WHEN**, le cas échéant, est évaluée à vrai.

34

Création de triggers PL/SQL

Les noms de corrélation

- Dans un trigger ligne, on doit pouvoir accéder aux ancienne et nouvelle valeurs de colonne de la ligne.
- Les noms de corrélation permettent de désigner ces deux valeurs : un nom pour l'ancienne et un pour la nouvelle.
  - La nouvelle valeur est appelée **:new.colonne**
  - L'ancienne valeur est appelée **:old.colonne**

- **Exemple :** IF : new.salaire < old.salaire ...
  - Si l'instruction de déclenchement du trigger est INSERT, seule la nouvelle valeur a un sens.
  - Si l'instruction de déclenchement du trigger est DELETE, seule l'ancienne valeur a un sens.

35

L'option REFERENCING

- Si une table s'appelle **NEW** ou **OLD**, on peut utiliser **REFERENCING** pour éviter l'ambiguïté entre le nom de la table et le nom de corrélation.

- **Exemple: soit la table New (Colonne1, colonne2)**

```
CREATE TRIGGER nomtrigger BEFORE UPDATE
ON  New REFERENCING New AS newnew
FOR EACH ROW
BEGIN
  :newnew.colonne1:= TO_CHAR(:newnew.colonne2);
END;
```

36

Les prédicats conditionnels

Inserting , Deleting et Updating

- Quand un trigger comporte plusieurs instructions de déclenchement (par exemple INSERT OR DELETE OR UPDATE), on peut utiliser des prédicats conditionnels (INSERTING, DELETING et UPDATING) pour exécuter des blocs de code spécifiques pour chaque instruction de déclenchement.

- Exemple: **Employes** (**NSS**, Nom, Prenom, NumService*) Service(**CodeServ**, Nom_service, NbEmp);

```
CREATE TRIGGER ... BEFORE INSERT OR UPDATE ON Employes
For each row
BEGIN ......
    IF INSERTING THEN  Update Service Set NbEmp= NbEmp+1 where CodeServ=:new. NumService   END IF;
    IF UPDATING THEN UPDATE Service SET NbEmp = NbEmp - 1 WHERE CodeServ = :old.NumService;
                     UPDATE Service SET NbEmp = NbEmp + 1 WHERE CodeServ = :new.NumService;
    END IF;
END;
```

37

Les prédicats conditionnels

- **UPDATING** peut être suivi d'un nom de colonne

Employes (NSS, Nom, Prenom, Salaire)

```
set serveroutput on;   --Activer le serveur d'affichage
CREATE OR REPLACE TRIGGER update_employe_trigger
AFTER UPDATE OF salaire ON employes
FOR EACH ROW
BEGIN
  IF UPDATING ('SALAIRE') THEN
     IF (:NEW.salaire) < (:OLD.salaire) THEN
            dbms_output.put_line('Le salaire de l employe '|| :NEW.Nom|| ':  ne change pas ');
     END IF;
  END IF;
END;
/
```

38

Manuel PL/SQL

- **BLOC PL/SQL**

[**DECLARE** … déclaration et initialisation]
BEGIN
… instructions exécutables
[**EXCEPTION** … interception des erreurs]
END;
Select * into v1
From etudiant
Where mat=10;

- **Exemple**

DECLARE
Mot **char(5);**
note **number(4,2):=10;**
x **number(4) :=0;**
v1 **table%ROWTYPE;** type du tuple d'une table
V1 ETUDIANT%ROWTYPE;
v2 **table.attribut%TYPE;** type d'un attribut d'une table
v2 ETUDIANT.Mat%TYPE;

39

Ordre de traitement des ligne

- On ne peut pas gérer l'ordre des lignes traitées par une instruction SQL.
- On ne peut donc pas créer un trigger qui dépende de l'ordre dans lequel les lignes sont traitées.
- **Triggers en cascade**
  - Un trigger peut provoquer le déclenchement d'un autre trigger.
  - ORACLE autorise jusqu'à 32 triggers en cascade à un moment donné

40

Activation d'un trigger

- Un trigger peut être activé ou désactivé.
- S'il est désactivé, ORACLE le stocke mais l'ignore.
- On peut désactiver un trigger si :
  - il référence un objet non disponible
  - on veut charger rapidement un volume de données important ou recharger des données déjà contrôlées.
- Par défaut, un trigger est activé dès sa création.

41

Activation d'un trigger

- Pour désactiver un trigger, on utilise l'instruction **ALTER TRIGGER** avec l'option **DISABLE** :
  ALTER TRIGGER nomtrigger **DISABLE**;
- On peut désactiver tous les triggers associés à une table avec la commande :
  ALTER TABLE nomtable **DISABLE ALL TRIGGERS**;
- A l'inverse on peut réactiver un trigger :
  ALTER TRIGGER nomtrigger **ENABLE**;
- ou tous les triggers associés à une table :
  ALTER TABLE nomtable **ENABLE ALL TRIGGERS**;

42

Recherche d'information sur les triggers

- Les définitions des triggers sont stockées dans les tables de la métabase, notamment dans les tables
  - **USER_TRIGGERS:** Cette vue contient des informations sur les triggers définis par l'utilisateur connecté
  - **ALL_TRIGGERS :** Cette vue contient des informations sur tous les triggers accessibles à l'utilisateur connecté, y compris ceux définis par d'autres utilisateurs. Cependant, seuls les objets auxquels l'utilisateur connecté a accès sont inclus dans cette vue.
  - **DBA_TRIGGERS:** Cette vue contient des informations sur tous les triggers de la base de données, quelle que soit l'utilisateur qui les a définis. Cette vue est accessible uniquement par les utilisateurs disposant du privilège DBA.

43

Gestion des exceptions

- Si une erreur se produit pendant l'exécution d'un trigger, toutes les mises à jour produites par le trigger ainsi que par l'instruction qui l'a déclenché sont défaites.
- On peut introduire des exceptions en provoquant des erreurs.
  - Une exception est une erreur générée dans une procédure PL/SQL.
  - Elle peut être prédéfinie ou définie par l'utilisateur.
  - Un bloc PL/SQL peut contenir un bloc EXCEPTION gérant les différentes erreurs possibles avec des clauses WHEN.
  - Une clause **WHEN OTHERS THEN ROLLBACK**; gère le cas des erreurs non prévues.

44

Exception prédéfinies

- **NO_DATA_FOUND** cette exception est générée quand un SELECT INTO ne retourne pas de lignes
- **DUP_VAL_ON_INDEX** tentative d'insertion d'une ligne avec une valeur déjà existante pour une colonne à index unique
- **ZERO_DIVIDE** division par zéro

45

Deux types d'exceptions

- **Exceptions SQL**
  - Déjà définies (pas de déclaration)
    - DUP_VAL_ON_INDEX
    - NO_DATA_FOUND
    - OTHERS
  - Non définies
    - Déclaration obligatoire avec le n° erreur (**sqlcode**)

```
nomerreur    EXCEPTION;
PRAGMA  EXCEPTION_INIT(nomerreur,    n°erreur);
```

46

Gestion des exceptions

```
DECLARE
  NomErreur EXCEPTION;
  PRAGMA EXCEPTION_INIT(NomErreur, -20001);
BEGIN
   ….
  RAISE_APPLICATION_ERROR(-20001, 'Une erreur s est produit lors du traitement');
  EXCEPTION
      WHEN NomErreur THEN
        DBMS_OUTPUT.PUT_LINE(' Erreur : ' || SQLCODE || ' - ' || sqlerrm );
      WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLCODE || ' - ' || sqlerrm);
END;
```

47

Gestion des Exceptions

Principe:

- Toute erreur (SQL ou applicative) entraîne automatiquement un débranchement vers le paragraphe EXCEPTION :

```
BEGIN
    instruction1;
    instruction2;
        ……
    instructionn;
EXCEPTION
    WHEN exception1  THEN    ……
    WHEN exception2  THEN    ……
    WHEN OTHERS      THEN
     ……
END;
```

Débranchement involontaire (erreur SQL) ou volontaire (erreur applicative)

48

Exemple

```
BEGIN
INSERT INTO Employes (NSS, Nom, Prenom, Date-rect)
VALUES (1001, 'Mohamed', 'Ahmed', TO_DATE('01-JAN-96', 'DD-MON-YYYY'));
UPDATE Departement SET ID_Responsable = 1001 WHERE ID_departement = 10;
DELETE FROM Employes WHERE NSS = 1002;
EXCEPTION
  WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Aucune donnée trouvée.')
  WHEN DUP_VAL_ON_INDEX THEN DBMS_OUTPUT.PUT_LINE('Violation d''index unique.');
  WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Une erreur s''est produite : ' || SQLERRM);
END;
```


Exemple 1

- **Exceptions SQL**
  - Exceptions applicatives
    - Déclaration sans n° erreur

```
nomerreur    EXCEPTION;
```

- **Exemple**

```
DECLARE
enfant_sans_parent  EXCEPTION;
PRAGMA EXCEPTION_INIT(enfant_sans_parent,-2291);
BEGIN
    INSERT INTO fils VALUES ( ……. );
    EXCEPTION
        WHEN enfant_sans_parent THEN
          ……
        WHEN OTHERS THEN ……
END;
```


Exemple2

```
set serveroutput on;
CREATE OR REPLACE TRIGGER update_employe_trigger
before UPDATE OF salaire ON employes
FOR EACH ROW
BEGIN
  IF UPDATING ('SALAIRE') THEN
     IF ((:NEW.salaire) < (:OLD.salaire)) THEN
                    RAISE_APPLICATION_ERROR(-20001, 'nouveau salaire < ancien salaire');
                    ELSE dbms_output.put_line(:NEW.Nom|| ': a un nouveau salaire');
     END IF;
  END IF;
END;
/
```

Langage PLSQL

52

Langage de bloc PL/SQL

- SQL : langage ensembliste
  - Ensemble de requêtes distinctes
  - Langage de 4ème génération : on décrit le résultat sans dire comment il faut accéder aux données
  - Obtention de certains résultats : encapsulation dans un langage hôte de 3ème génération

53

Langage de bloc PL/SQL

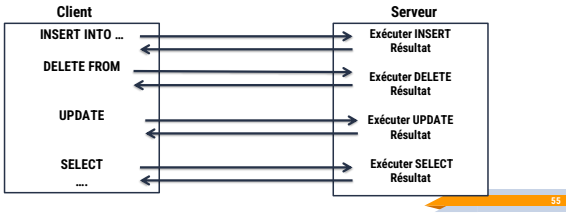
PL/SQL

- 'Procedural Language' : sur-couche procédurale à SQL, boucles, contrôles, affectations, exceptions, ….
  - Chaque programme est un bloc (BEGIN – END)
  - Programmation adaptée pour :
    - Transactions
    - Une architecture Client - Serveur

54

Requêtes SQL

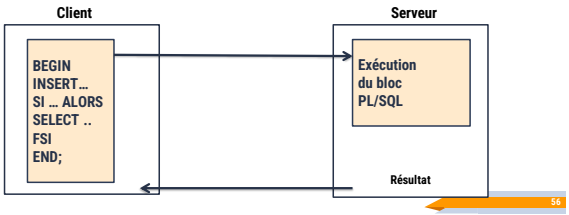
- Chaque requête 'client' est transmise au serveur de données pour être exécutée avec retour de résultats



55

Bloc PL/SQL

- Le bloc de requêtes est envoyé sur le serveur. Celui-ci exécute le bloc et renvoie 1 résultat final



56

Format d'un bloc PL/SQL

- **Section DECLARE : déclaration de**
  - Variables locales simples
  - Variables tableaux
  - cursors
- **Section BEGIN**
  - Section des ordres exécutables
  - Ordres SQL
  - Ordres PL
- **Section EXCEPTION**
  - Réception en cas d'erreur
  - Exceptions SQL ou utilisateur

```
DECLARE
--déclaration
BEGIN
-- exécutions
EXCEPTION
--erreur
END;
/
```

57

Variable simples

- Variables de type SQL

```
nbr                 NUMBER(2) ;
nom                 VARCHAR(30) ;
minimum CONSTANT    INTEGER := 5 ;
salaire             NUMBER(8,2) ;
debut               NUMBER NOT NULL ;
```

- Variables de type booléen (TRUE, FALSE, NULL)

```
fin        BOOLEAN ;
Reponse    BOOLEAN DEFAULT TRUE ;
ok         BOOLEAN := TRUE;
```

58

Variables faisant référence au dictionnaire de données

- **Référence à une colonne (table, vue)**

  vsalaire employe.salaire%TYPE;
  vnom etudiant.nom%TYPE;
  Vcomm vsalaire%TYPE;

- **Référence à une ligne (table, vue)**

  vemploye employe%ROWTYPE;
  vetudiant etudiant%ROWTYPE;

- **Variable de type 'struct'**
- Contenu d'une variable : **variable.colonne**

  vemploye.adresse

59

Instruction PL

- **Affectation (:=)**
  - A := B;
- **Structure alternative ou conditionnelle**
  - **Opérateurs SQL :** >, <, … ,OR, AND, …., BETWEEN, LIKE, IN
  - **IF …. THEN …. ELSE ……END IF;**

```
IF   condition     THEN
        instructions;
ELSE
        instructions;
        IF condition THEN instructions;
          ELSIF condition THEN instructions;
        ELSE instructions;
END IF;
```

60

Manuel PL/SQL

Si alors

```
IF condition
    THEN instruction;
END IF;
```

Si alors sinon

```
IF condition
    THEN instruction;
     ELSE  instruction;
END IF;
```

Imbrication de condition

```
IF condition  THEN instruction;
ELSEIF condition2  THEN instruction;
ELSEIF …
ELSE  instruction;
END IF;
```

61

Structure alternative : CASE (1)

- Choix selon la valeur d'une variable

```
CASE variable
    WHEN valeur1 THEN action1;

    WHEN valeur1 THEN action1;
    WHEN valeur2 THEN action2;
      ……
     ELSE action;
END CASE;
```

62

Structure alternative : CASE (2)

- Plusieurs choix possibles

```
CASE
    WHEN expression1 THEN  action1;
    WHEN expression2 THEN action2;
    ……
    ELSE action;
END CASE;
```

63

Manuel PL/SQL

```
CASE variable
   WHEN expr1 THEN instruction1;
   WHEN expr2 THEN instruction2; …
   WHEN exprN THEN instructionN;
[ELSE instructionN+1]
END CASE;
```

```
CASE
   WHEN condition1 THEN instruction1;
   WHEN conditionN THEN instruction2; …
   WHEN conditionN THEN instructionN;
[ELSE instructionN+1]
END CASE;
```

64

Manuel PL/SQL

- **Boucle POUR**

```
FOR indice IN borne1.. borne2LOOP
    instructions ;
END LOOP;
```

- **Boucle répéter**

```
LOOP
    instructions ;
    EXIT WHEN (condition);
END LOOP;
```

- **Boucle tant que**

```
WHILE (condition) LOOP
   instructions ;
END LOOP;
```

- **Affichage à l'ecran**

```
DBMS_OUTPUT.PUT_LINE (….|| ….|| …….);
```

65

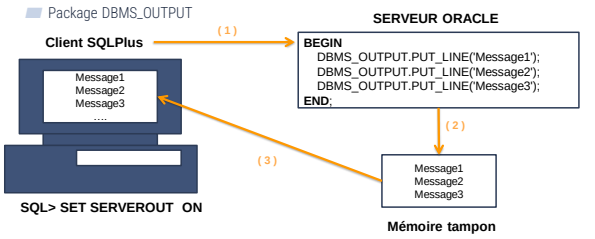
Affichage de résultats intermédiaires

- Package **DBMS_OUTPUT**
  - **Messages enregistrés** dans une mémoire tampon côté serveur
  - La mémoire tampon est affichée sur le poste client à la fin

66

Affichage de résultats intermédiaires

- Package DBMS_OUTPUT



Le package DBMS_OUTPUT

- **Écriture dans le buffer avec saut de ligne**
  - DBMS_OUTPUT.PUT_LINE(<chaîne caractères>);
- **Écriture dans le buffer sans saut de ligne**
  - DBMS_OUTPUT.PUT(<chaîne caractères>);
- **Écriture dans le buffer d'un saut de ligne**
  - DBMS_OUTPUT.NEW_LINE;

```
DBMS_OUTPUT.PUT_LINE('Affichage des n premiers ');
DBMS_OUTPUT.PUT_LINE('caractères en ligne ');
FOR i IN 1..n LOOP
   DBMS_OUTPUT.PUT(tab_cars(i));
END LOOP;
DBMS_OUTPUT.NEW_LINE;
```


Sélection mono – ligne: SELECT …. INTO

- Toute valeur de colonne est rangée dans une variable avec INTO

```
SELECT nom, adresse, tel INTO vnom, vadresse, vtel
FROM Employés WHERE NSS= matricule;
```

```
SELECT nom, adresse, nom_service INTO vnom, vadresse, vnom_serv
FROM Employés e, Service s
WHERE NSS=matricule  AND e.Num_serv=s.num_serv;
```

- Variable ROWTYPE

```
SELECT *  INTO vempl FROM Employés WHERE NSS=matricule;
…
DBMS_OUTPUT.PUT_LINE('Nom Employé : '|| vempl.nom);
```

Sélection multi – ligne: Les CURSEURS

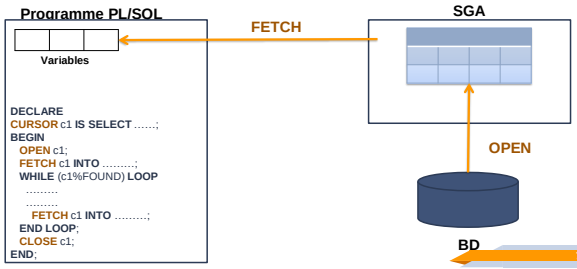
- **Principe des curseurs**
  - Obligatoire pour sélectionner plusieurs lignes
  - Zone mémoire (SGA : Share Global Area) partagée pour stocker les résultats
  - Le curseur contient en permanence l'@ de la ligne courante
  - **Curseur implicite**
    - **SELECT t.* FROM** table t **WHERE** ……
    - **t** est un curseur utilisé par SQL
  - **Curseur explicite**
    - DECLARE CURSOR

70

- **Curseur explicite**
  - **Déclaration du curseur : DECLARE**
    - Ordre SQL sans exécution
  - **Ouverture du curseur : OPEN**
    - SQL 'monte' les lignes sélectionnées en SGA
  - **Sélection d'une ligne : FETCH**
    - Chaque FETCH ramène une ligne dans le programme client
    - Tant que ligne en SGA : FETCH
  - **Fermeture du curseur : CLOSE**
    - Récupération de l'espace mémoire en SGA

71

Traitement d'un curseur



72

Gestion classique d'un curseur

```
DECLARE
   CURSOR c1 IS SELECT nom, moyenne FROM etudiant;
   vnom   etudiant.nom%TYPE;  vmoyenne   etudiant.moyenne%TYPE;
   e1 ,e2 NUMBER;
BEGIN
   OPEN c1;
   FETCH c1 INTO vnom,vmoyenne;
   WHILE c1%FOUND LOOP
     IF vmoyenne < 10 THEN e1:=e1+1; INSERT INTO liste_refus VALUES (vnom);
     ELSE e2:=e2+1; INSERT INTO liste_recus VALUES (vnom);   END IF;
     FETCH c1 INTO vnom,vmoyenne;
  END LOOP;
  CLOSE c1;
   DBMS_OUTPUT.PUT_LINE (TO_CHAR (e2) || 'Reçus ');
   DBMS_OUTPUT.PUT_LINE (TO_CHAR (e1) || 'Refus ');
   COMMIT;
END;
```

73

Les variables système des Curseurs

- **Curseur%FOUND**
  - Variable booléenne
  - Curseur toujours 'ouvert' (encore des lignes)
- **Curseur%NOTFOUND**
  - Opposé au précédent
  - Curseur 'fermé' (plus de lignes)
- **Curseur%COUNT**
  - Variable number
  - Nombre de lignes déjà retournées
- **Curseur%ISOPEN**
  - Booléen : curseur ouvert ?

74

Procédures & fonctions Stockées

75

Procédures Stockées

- **Fonctions**
  - Programme (PL/SQL) stocké dans la base
  - Le programme client exécute ce programme en lui passant des paramètres (par valeur)
    - Si le code est bon , le SGBD conserve le programme source (USER_SOURCE) et le programme compilé
  - Le programme compilé est optimisé en tenant compte des objets accélérateurs (INDEX, …)

76

Procédures Stockées

- Déclaration d'une procédure stockée

```
CREATE [OR REPLACE] PROCEDURE <nom_procédure>
[(variable1 type1, ..., variablen typen [OUT])] AS
...
-- déclarations des variables et
-- curseurs utilisées dans le corps de la procédure
BEGIN
....
-- instructions SQL ou PL/SQL
EXCEPTION
....
END;
/
```

77

Procédures Stockées

- Exemple: Inscription des étudiants

```
CREATE PROCEDURE inscription (ide varchar2(10),  pnom varchar2(30),
                              spec varchar2(30), ann_ins number)
AS
BEGIN
  DBMS_OUTPUT.PUT_LINE('Début inscription de ||pnom');
  INSERT INTO etudiant VALUES(ide,pnom,spec);
  INSERT INTO inscrire VALUES(ide,ann_ins);
  DBMS_OUTPUT.PUT_LINE('Transaction réussie');
COMMIT;
END;
/
```

78

Appel de la procédure

- A partir de sqlplus

```
ACCEPT vide PROMPT 'Entrer le matricule : '
……
EXECUTE inscription('&ide','&vnom','&an_ins', &spec');
```

- A partir de PL/SQL

```
inscription (ide,nom,an_ins, spec);
```


Les fonctions stockées

- Comme une procédure mais qui ne retourne qu'un seul résultat
- Même structure d'ensemble qu'une procédure
- Utilisation du mot clé RETURN pour retourner le résultat
- Appel possible à partir de :
  - Une requête SQL normale
  - Un programme PL/SQL
  - Une procédure stockée ou une autre fonction stockée

Déclaration d'une fonction stockée

```
CREATE [OR REPLACE] FUNCTION nom_fonction
[(paramètre1 type1, …… , paramètren typen)]
RETURN type_résultat IS
-- déclarations de variables,curseurs et exceptions
BEGIN
30
BEGIN
-- instructions PL et SQL
RETURN(variable);
END;
/
```

1 ou plusieurs RETURN

Exemple de fonction stockée

```
CREATE OR REPLACE FUNCTION moy_points_marques (eqj joueur.ideq%TYPE)
RETURN NUMBER IS   moyenne_points_marques NUMBER(4,2);
BEGIN
   SELECT AVG(totalpoints) INTO moyenne_points_marques
   FROM joueur WHERE ideq=eqj;
   RETURN(moyenne_points_marques);
END;
/
```


Appel d'une fonction

- A partir d'une requête SQL

```
SELECT moy_points_marques ('e1') FROM dual;
```

```
SELECT   nomjoueur   FROM   joueur
WHERE  totalpoints > moy_points_marques ('e1');
```

- A partir d'une procédure ou fonction

```
BEGIN
……
IF moy_points_marques (equipe) > 20 THEN ……
END;
```